

INTRODUCTION TO DATA SCIENCE



Predicting Depression and Mental Health Risk through Social Media Posts

Semester Project Report
BS(CS)-6B

Team Members

Aqsa Riaz (01-134231-100)
Syeda Maleeha Bano Naqvi (01-134231-092)

Submitted To:

Sir Mohammad Faizan

Department of Computer Science

BAHRIA UNIVERSITY, ISLAMABAD

CONTENTS

1. Project Description.....	4
2. Abstract.....	4
3. Introduction.....	5
3.1. Problem Background.....	5
3.2. Why Depression Detection is Important.....	5
3.3. Why the Dataset is Suitable.....	6
3.4. Methodology.....	6
4. Scope	7
5. Methodology.....	7
5.1. Data Collection.....	8
5.1.1.1. Justification for Dataset Selection.....	8
5.2. Data Preprocessing.....	8
5.2.1.1. Checking for Missing or Inconsistent Data.....	8
5.2.1.2. Cleaning and Normalization Text.....	8
5.2.1.3. Lemmatization.....	9
5.2.1.4. Feature Engineering.....	9
5.2.1.5. Handling Class Imbalance.....	9
5.2.1.6. Dataset Splitting.....	11
5.2.1.7. Feature Scaling and Normalization.....	11
5.3. Exploratory Data Analysis (EDA).....	11
5.3.1.1. Descriptive Statistics.....	11
5.3.1.2. Visualizations.....	11
5.3.1.3. Insights Extracted.....	11
5.4. Modeling / Pattern Extraction.....	11
5.4.1.1. Model Selection.....	12
5.4.1.2. Model Training.....	12
5.4.1.3. Model Evaluation.....	12
5.4.1.4. Testing on new posts.....	12
5.4.1.5. Model Persistence.....	13
6. Advantages & Limitations.....	13
7. Applications.....	13
8. Future Enhancements	14

9. Flowchart.....	16
10. Machine Learning and Backend Code.....	16
10.1. train_model.py.....	16
10.2. app.py.....	18
10.3. custom_transformers.py.....	18
10.4. fetch_tweets.py.....	19
11. Flutter Application Code (frontend GUI).....	19
11.1. main.dart.....	19
11.2. .env.....	28
11.3. pubspec.yaml.....	27
12. Output.....	28

Predicting Depression and Mental Health Risk through Social Media Posts

1. Project Description

Mental health challenges such as depression and anxiety have become global concerns, particularly in the digital era where individuals often express their emotions through social media. Platforms like Reddit, Twitter, and Facebook serve as open spaces where users share their experiences, moods, and struggles. Analyzing this online data through data science techniques provides opportunities to understand emotional patterns and detect signs of mental distress.

This project applies machine learning and natural language processing (NLP) to analyze social media posts and predict the likelihood of depressive tendencies. Using the Kaggle Depression and Mental Health Dataset, which contains real user posts labeled as depressed or non-depressed, the project follows the complete data science pipeline — from preprocessing and exploratory data analysis to model training and evaluation. The data preprocessing stage involves cleaning the text, removing stop words, tokenizing, and applying lemmatization to prepare it for analysis. Exploratory Data Analysis (EDA) will include visualizations such as word clouds, sentiment distributions, and correlation heatmaps to uncover meaningful linguistic and emotional patterns.

For the machine learning phase, models like Logistic Regression, Random Forest, and Naïve Bayes will be trained to classify posts accurately. Evaluation metrics including Accuracy, Precision, Recall, F1-Score, and ROC-AUC will be used to assess performance, with special focus on minimizing false negatives, as undetected depressive signals can lead to real-world mental health risks.

The final output will be a data-driven prototype that predicts the likelihood of depression in a given text input. If time permits, an interactive dashboard will be developed to visualize emotional trends and model predictions. This project not only strengthens academic understanding of data science concepts but also demonstrates how AI and analytics can contribute to social well-being.

2. Abstract

Depression is one of the most pressing public health concerns of the 21st century, and early detection is critical for prevention and intervention. In the digital age, people frequently express their feelings and struggles through social media posts, making these platforms valuable sources for analyzing emotional well-being.

This project aims to develop a machine learning-based approach to identify potential signs of depression in social media text data. Using the Kaggle Depression and Mental Health Dataset, the project involves text preprocessing, feature extraction, exploratory data analysis, and model development. Techniques such as TF-IDF vectorization and Natural Language Processing (NLP) are

applied to convert unstructured text into quantitative features.

Predictive models like Logistic Regression and Random Forest will be implemented to classify posts as depressed or non-depressed. Evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC will be used to assess the models' effectiveness. The expected outcome is a prototype capable of predicting depressive tendencies from user-generated text with high reliability. This project highlights how data science can be applied to social good, offering insights into digital mental health trends and encouraging ethical use of AI in analyzing sensitive human data.

3. Introduction

Depression is a growing global concern, and early detection is essential. Social media posts often reveal users' emotions, providing a source for analysis. This project applies data science and machine learning to predict depressive tendencies from social media text using a Kaggle dataset. The workflow includes data preprocessing, NLP-based feature extraction, exploratory analysis, and model training (Logistic Regression, Random Forest). The goal is to develop a prototype that identifies depressive patterns while ensuring ethical AI practices and user privacy.

3.1. Problem Background

Depression and other mental health issues have seen a dramatic increase worldwide, impacting millions of individuals across age groups. In recent years, social media has become a major platform for self-expression, where people often post about their emotions, stress, and personal experiences. While these posts may appear casual, they frequently contain linguistic cues that reveal underlying psychological states.

Traditional methods for diagnosing depression rely on clinical interviews and self-reported surveys. However, such methods are limited by access, stigma, and time constraints. Data science offers a scalable, automated, and non-intrusive approach to detecting emotional distress by analyzing the digital footprints users leave behind online.

The key challenge lies in processing unstructured textual data and differentiating between casual emotional expression and potential signs of depression. This project addresses that challenge using Natural Language Processing (NLP) to extract patterns from text and machine learning algorithms to classify posts. The Kaggle dataset chosen for this project provides a realistic foundation with labeled examples of both depressed and non-depressed posts, enabling the development of predictive models that can generalize effectively.

3.2. Why Depression Detection is Important

Depression affects individuals' productivity, relationships, and overall well-being, and it

remains one of the leading causes of disability worldwide. Early detection is essential for timely intervention and support. With the rise of online communication, AI-based detection tools can help mental health professionals and organizations identify at-risk individuals through their digital expressions.

The significance of this project extends beyond technical innovation — it serves a social purpose. By using data science for emotional insight, this project promotes awareness of mental health and responsible technology use. Moreover, analyzing social media data allows for real-time detection of mental health risks, which could be integrated into support systems or awareness campaigns.

3.3. Why the Dataset is Suitable

The Kaggle Depression and Mental Health Dataset is ideal for this project because it contains a large number of labeled social media posts collected from users discussing their emotions and experiences. The dataset provides both depressed and non-depressed categories, enabling the supervised learning approach required for model training.

The data is anonymized to ensure user privacy and includes diverse language structures typical of real-world social media communication. Its natural text variability allows for testing different preprocessing and NLP techniques. Additionally, it has been widely used in academic and applied AI research, making it a reliable benchmark for evaluating depression detection models.

3.4. Methodology

The project follows a structured data science workflow consisting of the following stages:

- **Data Preprocessing:** Cleaning text data, removing noise (e.g., URLs, punctuation), tokenizing words, and applying lemmatization.
- **Exploratory Data Analysis (EDA):** Visualizing patterns through word clouds, frequency plots, and sentiment graphs to understand emotional trends.
- **Feature Extraction:** Using TF-IDF Vectorization to convert text into numerical representations suitable for machine learning.
- **Model Development:** Implementing and comparing Logistic Regression, Random Forest, and Naïve Bayes classifiers to predict depression likelihood.
- **Model Evaluation:** Measuring model performance using Accuracy, Precision, Recall, F1-

Score and ROC-AUC.

- **Insights and Reporting:** Presenting key findings, patterns, and visualizations dashboards that summarize results.

The methodology ensures both technical rigor and ethical responsibility, demonstrating how data science can be applied meaningfully to address social and psychological issues.

4. Scope

This project focuses on leveraging Natural Language Processing (NLP) and machine learning to detect depressive tendencies from social media posts. Its scope includes:

4.1. Text-Based Mental Health Detection

Analyzing user-generated content from platforms like Twitter, Reddit, Facebook, and other online forums to identify patterns indicating emotional distress or depressive symptoms.

4.2. End-to-End Data Science Pipeline

The project covers the full pipeline:

- Data preprocessing
- Feature Engineering
- Exploratory Data Analysis
- ML model Development
- Evaluation and prediction
- Frontend application for real-time input

4.3. Multi-Model Comparison

The project explores and compares different algorithms including Logistic Regression, Naïve Bayes, and Random Forest. This helps determine which model performs best for short social media text.

4.4. Real-Time Prediction Prototype

A working backend–frontend system is developed to allow users to input text and receive depression-risk predictions instantly.

4.5. Practical and Ethical Applications

The project stays within ethical guidelines by using anonymized public data and focusing on awareness, not clinical diagnosis.

5. Methodology Details

5.1.Data Collection

- **Source:** Kaggle dataset titled “Mental Health Twitter Dataset.”
- **Contents:**
 - Post_text: the text of each tweet.
 - Label: binary label indicating whether the post is depressive (1) or not (0).
- **Rationale for dataset choice:**
 - Real-world dataset with social media posts, making it ideal for detecting emotional and psychological states.
 - Provides labeled examples for supervised classification tasks.
 - Short textual data enables testing NLP preprocessing techniques, including cleaning, lemmatization, and feature extraction.
- **Volume:** The dataset contains thousands of tweets, sufficient to train and test machine learning models reliably

5.1.1.1. Justification for Dataset Selection

- Twitter users often express raw emotional content, which is valuable for detecting mental health patterns.
- The dataset’s labeling allows for supervised learning, enabling direct model training.
- Using social media data ensures the model is applicable to real-time, unstructured text, which is a common challenge in NLP applications.

5.2.Data Preprocessing

Preprocessing is a crucial step in NLP because **raw text contains noise** that can degrade model performance.

5.2.1.1. Checking for Missing or Inconsistent Data

- Functions like `df.info()` and `df.isnull().sum()` were used to inspect:
 - Missing values in the `post_text` column
 - Invalid data types (ensuring all text is string type)
- Findings: Minimal missing data, which was handled by type conversion or exclusion.

5.2.1.2. Cleaning and Normalization Text

- **Lowercasing:** Ensures that 'Sad' and 'sad' are treated the same.
- **Removing URLs:** Regex `r'http\S+|www\S+'` removes links that do not contribute to text sentiment.
- **Removing mentions (@username):** Regex `r'@\w+'` removes social media mentions.
- **Removing hashtags (# symbol):** Only removes the symbol, keeping the keyword.
- **Removing punctuation and special characters:** Regex `r'^a-z0-9\s'` removes noise.
- **Removing extra whitespace:** Ensures consistent tokenization.

Example:

Original: "Feeling sad today... #depressed http://t.co/link"

Cleaned: "feeling sad today depressed"

5.2.1.3. Lemmatization

- **Lemmatization:** Converts words to their dictionary form using spaCy, reducing feature sparsity.
- **Stopwords removal:** Words like 'the', 'is', and 'and' are removed as they carry little meaning.
- Added lemma column for lemmatized text.

Example:

"Feeling hopeless and sad" → "feel hopeless sad"

5.2.1.4. Feature Engineering

- **Text length metrics:**
 - Original text length (`text_length`)
 - Cleaned text length (`clean_length`)
 - Histograms show most tweets are <200 characters.
- **Word Class:**
 - Depressed posts: Words like "sad", "alone", "feel", "life" dominate.
 - Non-depressed posts: Words like "happy", "friends", "fun" dominate.
- **Sentiment Score:**
 - Using **TextBlob**, polarity scores [-1, 1] capture emotional content of text.
 - Positive sentiment scores override prediction in the pipeline if above threshold (>0.3).

5.2.1.5. Handling Class Imbalance

- Stratified splitting ensures train and test sets maintain the same proportion of depressed and non-depressed tweets.
- Logistic Regression trained with `class_weight='balanced'` to reduce bias towards the majority class.

5.2.1.6. Dataset Splitting

- Split dataset into 80% training and 20% testing using `train_test_split()` with `stratify=y`.
- Ensures evaluation reflects real-world distribution.

5.2.1.7. Feature Scaling and Normalization

- TF-IDF features normalized automatically.
- Sentiment scores scaled using `StandardScaler` to match the feature range of TF-IDF.
- Combined features allow models to capture both semantic patterns and emotional tone.

5.3.Exploratory Data Analysis (EDA)

5.3.1.1. Descriptive Statistics

- **Columns:** `post_text`, `label`, `clean_text`, `lemma`, `text_length`, `clean_length`.
- **Insights:**
 - Depressed tweets slightly shorter after cleaning due to removal of URLs/mentions.
 - Tweets contain a variety of vocabulary, requiring TF-IDF for effective feature extraction.

5.3.1.2. Visualizations

- **Label Distribution:** Countplot shows roughly balanced classes.
- **Tweet Lengths:** Histograms indicate most posts are concise (<200 characters).
- **Word Clouds:** Highlight the most common words per class, giving insight into emotional content.

5.3.1.3. Insights Extracted

- Depressed tweets often contain words expressing sadness, loneliness, hopelessness.
- Non-depressed tweets often contain social or positive words, indicating engagement and happiness.
- Combining text features with sentiment scores improves predictive power.

5.4.Modeling / Pattern Extraction

5.4.1.1. Model Selection

- Three models used for evaluation:
 - Logistic Regression: Baseline classifier; interpretable coefficients.
 - Multinomial Naive Bayes: Effective for word-count features (TF-IDF).
 - Random Forest Classifier: Captures non-linear interactions in features.
- Additionally, implemented a custom pipeline:
 - Combines TF-IDF features and sentiment scores
 - Logistic Regression as classifier
 - Supports sentiment override for strongly positive tweets.

5.4.1.2. Model Training

- TF-IDF vectorization on training set (max_features=10000, n-grams 1-2).
- Sentiment features calculated using TextBlob polarity scores.
- Features combined using a custom CombineFeatures transformer.
- Models trained using fit() on training data.

5.4.1.3. Model Evaluation

- Metrics used:
 - Accuracy: Correct Predictions/ total Predictions
 - Precision: True positive / predicted positives
 - Recall: True positives / actual positives
 - F1-Score: Harmonic mean of precision and recall
 - ROC-AUC: Area under ROC curve (if probability scores available)
- Confusion matrices provide insight into misclassification patterns.
- Logistic Regression coefficients allow identification of words highly indicative of depression.
- **Example Coefficients:**
 - Top positive words (depression): ['sad', 'alone', 'life', 'feel', 'depressed']
 - Top negative words (non-depression): ['happy', 'friends', 'fun', 'love', 'great']

5.4.1.4. Testing on new posts

- **Example predictions:**

- "I feel so sad and hopeless today..." → Depressed
- "Had a great day with my friends, feeling happy!" → Not Depressed
- Sentiments override ensures posts with strongly positive sentiment are predicted as Not depressed, reducing false positives.

5.4.1.5. Model Persistence

- Pipelines saved using joblib:
 - TF-IDF + Logistic Regression
 - Full pipeline with sentiment features and override logic.
- Enables real-time predictions without retraining.

6. Advantages and Limitations

Sr.No	Advantages (pros)	Limitations (cons)
1	Early Risk Identification – Helps flag depressive tendencies from text before symptoms worsen.	Not a Clinical Diagnosis – Cannot replace psychologists or professional medical evaluation.
2	Real-Time Predictions – The model can instantly classify new posts or user inp	Risk of Misclassification – Sarcasm, slang, or indirect emotional expression may cause false positives/negatives.
3	Scalable – Capable of analyzing thousands of posts automatically without human intervention.	Dataset Bias – Kaggle dataset may not represent all cultures, languages, or writing styles.
4	Uses NLP + Sentiment Analysis – Combining linguistic features with sentiment polarity improves accuracy.	Limited to Text Only – Does not consider tone, facial expressions, behavior, or medical history.
5	Non-Intrusive – Users don't need to take surveys; analysis is based on natural writing.	Privacy Concerns – Monitoring social media posts raises ethical considerations regarding user consent.
6	Cost-Effective – AI-based automation reduces the need for constant human monitoring.	Emotion Expression Varies – Some people hide emotions or express them in ways the model may not detect.
7	Interpretable for Researchers – Logistic Regression coefficients show which words are associated with depression.	Language Dependency – Works best for English; limited performance on mixed or regional languages.

7. Applications

- **Mental Health Support Platforms**

The system can be integrated into online counseling apps and mental health websites to automatically screen user text and identify early signs of emotional distress. This helps professionals prioritize users who may need immediate support.

➤ **Educational Institutions**

Universities and colleges can use this model in student help portals or anonymous feedback systems to detect stress, anxiety, or depressive expressions among students, allowing timely intervention by counselors.

➤ **Workplace Wellness Programs**

Organizations can use the system (anonymously and ethically) to analyze employee feedback or surveys and detect patterns of burnout, emotional fatigue, or declining morale.

➤ **Social Media Monitoring Tools**

Social media platforms or analytics tools can use the model to flag posts that indicate harmful or depressive tendencies. This can help provide users with automated support options or helplines.

➤ **Mental Health Chatbots**

AI chatbots can integrate the model to better understand user mood and respond with more empathetic or supportive messages, especially when detecting negative sentiment or depressive patterns.

➤ **Public Health and Nonprofit Organizations**

Agencies can analyze large-scale social media data to understand mental health trends in the population, helping them design awareness campaigns or support programs based on real-time emotional patterns.

8. Future Enhancements

➤ **Deep Learning Models (LSTM, BiLSTM, Transformers)**

Implement more advanced models like BERT, RoBERTa, or GPT-based classifiers for better understanding of context, sarcasm, and subtle expressions.

➤ **Multilingual Support**

Add processing for Urdu, Hindi, Arabic, and other regional languages for broader applicability in South Asian contexts.

➤ **Multi-Modal Depression Detection**

Include additional signals such as:

- Images
- Emojis
- Voice notes
- Interaction patterns

This can significantly improve accuracy.

➤ **Personalized Mental Health Insights**

Allow users to track their emotional patterns over time through dashboards and trend charts.

➤ **Real-World Deployment with APIs**

Deploy the model on cloud services (AWS, Azure, Firebase) and integrate with mobile or web apps for real-time screening.

➤ **Integration with Human Counselors**

Use the model as a first-line filter that alerts mental health experts when high-risk posts are detected.

➤ **Explainability & Transparency**

Improve interpretability using SHAP or LIME to show which words contributed to a high depression score.

➤ **Ethical Framework Development**

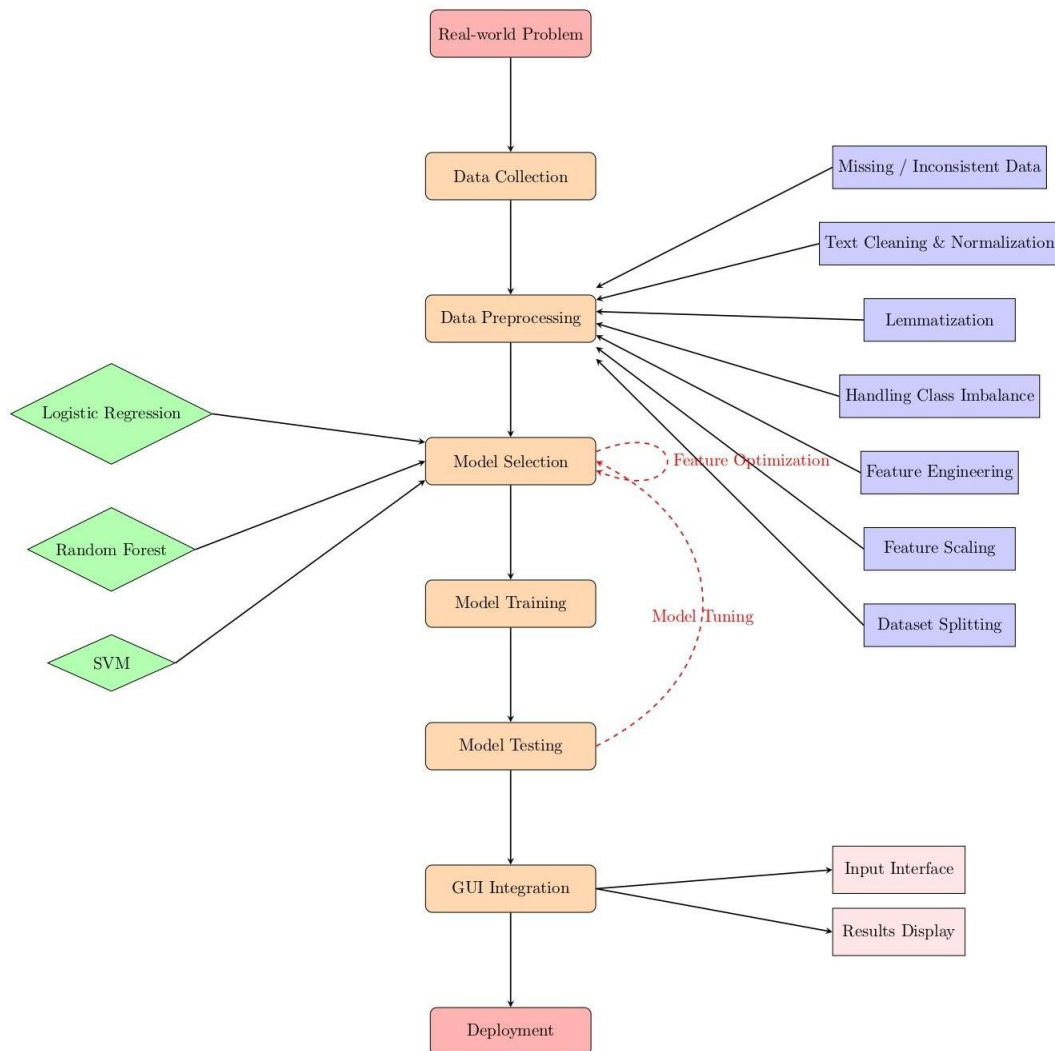
Ensure safe and responsible use by creating:

- Clear consent policies
- Data anonymization
- Bias auditing
- Non-invasive monitoring practices

➤ **Larger and More Diverse Datasets**

Training on diverse social media datasets (e.g., Reddit, Instagram captions, YouTube comments) will improve generalizability.

9. Flowchart



10. Machine Learning and Backend Code

10.1. train_model.py

```

1  import os
2  import pandas as pd
3  import numpy as np
4  import joblib
5  from sklearn.pipeline import Pipeline
6  from sklearn.feature_extraction.text import TfidfVectorizer
7  from sklearn.linear_model import LogisticRegression
8  from custom_transformers import SentimentExtractor, CombineFeatures
9  from scipy.sparse import hstack
10 from textblob import TextBlob
11 import nltk
12
13 nltk.download('brown')
14 nltk.download('punkt')
15 nltk.download('averaged_perceptron_tagger')
16 nltk.download('wordnet')
17
18 df = pd.read_csv(r"C:\Users\Aqsa\Desktop\6 semester\IDS\Mental-Health-Twitter.csv")
19
20 tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1,2))
21 sentiment = SentimentExtractor()
22
23 pipeline = Pipeline([
24     ('features', CombineFeatures(tfidf=tfidf, sentiment=sentiment)),
25     ('classifier', LogisticRegression(max_iter=1000, class_weight='balanced'))
26 ])
27
28 X = df['post_text']
29 y = df['label']
30
31 pipeline.fit(X, y)
32 print("Pipeline trained successfully!")
33
34 def predict_with_sentiment_override(pipeline, posts, threshold=0.3):
35     """
36     Predict depression using the pipeline.
37     Overrides prediction to 'Not Depressed' if sentiment > threshold.
38     """
39     predictions = pipeline.predict(posts)
40     probabilities = pipeline.predict_proba(posts)[:, 1]
41
42     results = []
43     for post, pred, prob in zip(posts, predictions, probabilities):
44         sentiment_score = TextBlob(post).sentiment.polarity
45         if sentiment_score > threshold: # strong positive sentiment
46             pred = 0 # mark as Not Depressed
47             prob = max(prob, 1 - sentiment_score)
48             label = "Depressed" if pred == 1 else "Not Depressed"
49             results.append((post, label, prob, sentiment_score))
50     return results
51
52 new_posts = [
53     "I feel happy today!",
54     "Feeling very sad...",
55     "Had a great day with my friends, feeling happy!",
56     "I feel so sad and hopeless today..."
57 ]
58
59 results = predict_with_sentiment_override(pipeline, new_posts)
60 for post, label, prob, sentiment in results:
61     print(f"Post: {post}\nPrediction: {label}, Probability: {prob:.2f}, Sentiment: {sentiment:.2f}\n")
62
63 os.makedirs("models", exist_ok=True)
64 joblib.dump(pipeline, "models/depression_detection_pipeline_sentiment_override.joblib")
65 print("Pipeline saved successfully at 'models/depression_detection_pipeline_sentiment_override.joblib'")
66

```

10.2. app.py

```

1  from flask import Flask, request, jsonify
2  import joblib
3  from flask_cors import CORS
4  from custom_transformers import SentimentExtractor, CombineFeatures
5
6  app = Flask(__name__)
7  CORS(app)
8
9  model_path = "models/depression_detection_pipeline_sentiment_override.joblib"
10 pipeline = joblib.load(model_path)
11 print("Pipeline loaded successfully!")
12
13 @app.route("/predict", methods=["POST"])
14 def predict():
15     data = request.get_json()
16     if not data or "posts" not in data:
17         return jsonify({"error": "No posts provided"}), 400
18
19     posts = data["posts"]
20
21     if isinstance(posts, str):
22         posts = [posts]
23     elif not isinstance(posts, list):
24         return jsonify({"error": "Posts must be a string or List of strings"}), 400
25     predictions = pipeline.predict(posts)
26     probabilities = pipeline.predict_proba(posts)[: , 1]
27
28     threshold = 0.7
29
30     results = []
31     for post, prob in zip(posts, probabilities):
32         pred_label = "Depressed" if prob > threshold else "Not Depressed"
33         results.append({
34             "post": post,
35             "prediction": pred_label,
36             "probability": float(prob)
37         })
38
39     return jsonify(results)
40
41 if __name__ == "__main__":
42     app.run(host="0.0.0.0", port=5000, debug=True)

```

10.3. custom_transformers.py

```

1  import numpy as np
2  from sklearn.base import BaseEstimator, TransformerMixin
3  from sklearn.feature_extraction.text import TfidfVectorizer
4  from sklearn.preprocessing import StandardScaler
5  from scipy.sparse import hstack
6  from textblob import TextBlob
7
8  class SentimentExtractor(BaseEstimator, TransformerMixin):
9      def fit(self, X, y=None):
10         return self
11      def transform(self, X):
12         sentiment_scores = [TextBlob(str(text)).sentiment.polarity for text in X]
13         return np.array(sentiment_scores).reshape(-1, 1)
14
15  class CombineFeatures(BaseEstimator, TransformerMixin):
16      def __init__(self, tfidf, sentiment):
17         self.tfidf = tfidf
18         self.sentiment = sentiment
19         self.scaler = StandardScaler()
20      def fit(self, X, y=None):
21         self.tfidf.fit(X)
22         sentiment_features = self.sentiment.transform(X)
23         self.scaler.fit(sentiment_features)
24         return self
25      def transform(self, X):
26         if isinstance(X, str):
27             X = [X]
28         tfidf_features = self.tfidf.transform(X)
29         sentiment_features = self.scaler.transform(self.sentiment.transform(X))
30         return hstack([tfidf_features, sentiment_features])
31

```

10.4. fetch_tweets.py

```

1  import tweepy
2
3
4  bearer_token = "AAAAAAAAAAAAAAAAAGBm5gEAAAAA4dfVmtMusKIXH2D7UoNr8pjY3qo%3DdpIL6rtXVHxSNhUED5jJdVfm4fg9IJ3awVwP5sKTPhcso5Duwi"
5  client = tweepy.Client(bearer_token=bearer_token)
6
7  user = client.get_user(username="nasa")
8  user_id = user.data.id
9  print(user_id)
10

```

11.Flutter Application Code (frontend GUI)

11.1. main.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_dotenv/flutter_dotenv.dart';
3  import 'package:http/http.dart' as http;
4  import 'dart:convert';
5
6  Future<void> main() async {
7    WidgetsFlutterBinding.ensureInitialized();
8    await dotenv.load(fileName: "assets/.env");
9    runApp(MyApp());
10 }
11
12 class MyApp extends StatelessWidget {
13   @override
14   Widget build(BuildContext context) {
15     return MaterialApp(
16       debugShowCheckedModeBanner: false,
17       title: 'Twitter Depression Predictor',
18       theme: ThemeData(
19         textTheme: Typography.blackMountainView,
20         fontFamily: 'Roboto',
21         colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
22       ), // ThemeData
23       home: TwitterPage(),
24     ); // MaterialApp
25   }
26 }
27
28 class TwitterPage extends StatefulWidget {
29   @override
30   TwitterPageState createState() => _TwitterPageState();
31 }
32
33 class _TwitterPageState extends State<TwitterPage> {
34   final TextEditingController _twitterController = TextEditingController();
35   bool isLoading = false;
36   String statusMessage = "";
37   List<Map<String, dynamic>> predictions = [];
38
39   Future<void> fetchAndPredictTweets() async {
40     final username = _twitterController.text.trim();
41     if (username.isEmpty) {
42       setState(() => statusMessage = "Enter a Twitter username.");
43       return;
44     }

```

```

45
46     setState(() {
47         isLoading = true;
48         statusMessage = "Fetching tweets...";
49         predictions = [];
50     });
51
52     try {
53         final bearerToken = dotenv.env['TWITTER_BEARER_TOKEN'] ?? '';
54         final flaskApiUrl = dotenv.env['API_URL'] ?? '';
55
56         if (bearerToken.isEmpty || flaskApiUrl.isEmpty) {
57             setState(() {
58                 statusMessage = "Environment variables not found!";
59             });
60             return;
61         }
62
63         final userResponse = await http.get(
64             Uri.parse('https://api.twitter.com/2/users/by/username/$username'),
65             headers: {"Authorization": "Bearer $bearerToken"},
66         );
67
68         if (userResponse.statusCode != 200) {
69             setState(() {
70                 statusMessage = "Error fetching user: ${userResponse.body}";
71             });
72             return;
73         }
74
75         final userData = json.decode(userResponse.body);
76         final userId = userData['data']['id'];
77
78         final tweetsResponse = await http.get(
79             Uri.parse(
80                 'https://api.twitter.com/2/users/$userId/tweets?max_results=5&tweet.fields=text',
81             ),
82             headers: {"Authorization": "Bearer $bearerToken"},
83         );
84
85         if (tweetsResponse.statusCode != 200) {
86             setState(() {

```

```

87 |         statusMessage = "Error fetching tweets: ${tweetsResponse.body}";
88 |     });
89 |     return;
90 | }
91 |
92 | final tweetsData = json.decode(tweetsResponse.body);
93 | List<String> tweets = [];
94 | if (tweetsData['data'] != null) {
95 |     tweets = List<String>.from(tweetsData['data'].map((t) => t['text']));
96 | }
97 |
98 | if (tweets.isEmpty) {
99 |     setState(() {
100 |         statusMessage = "No tweets found for @$username";
101 |     });
102 |     return;
103 | }
104 |
105 | final body = json.encode({"posts": tweets});
106 |
107 | final response = await http.post(
108 |     Uri.parse(flaskApiUrl),
109 |     headers: {"Content-Type": "application/json"},
110 |     body: body,
111 | );
112 |
113 | if (response.statusCode != 200) {
114 |     setState(() {
115 |         statusMessage = "Error from API: ${response.body}";
116 |     });
117 |     return;
118 | }
119 |
120 | final result = json.decode(response.body) as List<dynamic>;
121 | setState(() {
122 |     predictions = result.cast<Map<String, dynamic>>();
123 |     statusMessage = "Predictions fetched successfully! ✅";
124 | });
125 | } catch (e) {
126 |     setState(() {
127 |         statusMessage = "Error: $e";

```

```

128     });
129   } finally {
130     setState(() => isLoading = false);
131   }
132 }
133
134 @override
135 Widget build(BuildContext context) {
136   return Scaffold(
137     backgroundColor: Color(0xFF3F0FF),
138     body: SafeArea(
139       child: SingleChildScrollView(
140         padding: EdgeInsets.all(20),
141         child: Column(
142           children: [
143             // HEADER CARD
144             Container(
145               width: double.infinity,
146               padding: EdgeInsets.all(25),
147               decoration: BoxDecoration(
148                 gradient: LinearGradient(
149                   colors: [Colors.deepPurple, Colors.deepPurpleAccent],
150                   begin: Alignment.topLeft,
151                   end: Alignment.bottomRight,
152                 ), // LinearGradient
153                 borderRadius: BorderRadius.circular(20),
154                 boxShadow: [
155                   BoxShadow(
156                     blurRadius: 10,
157                     spreadRadius: 1,
158                     color: Colors.deepPurple.withOpacity(0.3),
159                   ), // BoxShadow
160                 ],
161               ), // BoxDecoration
162             child: Column(
163               children: [
164                 Icon(Icons.monitor_heart, size: 80, color: Colors.white),
165                 SizedBox(height: 10),
166                 Text(
167                   "Twitter Depression Predictor",
168                   textAlign: TextAlign.center,

```

```

169         style: TextStyle(
170           fontSize: 26,
171           color: Colors.white,
172           fontWeight: FontWeight.bold,
173         ), // TextStyle
174       ), // Text
175       SizedBox(height: 5),
176       Text(
177         "Analyze emotional patterns from recent tweets",
178         style: TextStyle(fontSize: 14, color: Colors.white70),
179       ), // Text
180     ],
181   ), // Column
182 ), // Container
183
184   SizedBox(height: 25),
185
186   // INPUT FIELD
187   Container(
188     padding: EdgeInsets.all(15),
189     decoration: BoxDecoration(
190       color: Colors.white,
191       borderRadius: BorderRadius.circular(18),
192       boxShadow: [
193         BoxShadow(
194           color: Colors.black12,
195           blurRadius: 8,
196           spreadRadius: 1,
197         ), // BoxShadow
198       ],
199     ), // BoxDecoration
200     child: Column(
201       children: [
202         TextField(
203           controller: _twitterController,
204           decoration: InputDecoration(
205             prefixIcon: Icon(
206               Icons.person,
207               color: Colors.deepPurple,
208             ), // Icon
209             labelText: "Enter Twitter Username",
210             labelStyle: TextStyle(color: Colors.deepPurple),

```

```

211         border: OutlineInputBorder(
212           borderRadius: BorderRadius.circular(12),
213         ), // OutlineInputBorder
214         focusedBorder: OutlineInputBorder(
215           borderRadius: BorderRadius.circular(12),
216           borderSide: BorderSide(
217             color: Colors.deepPurple,
218             width: 2,
219           ), // BorderSide
220         ), // OutlineInputBorder
221       ), // InputDecoration
222     ), // TextField
223     SizedBox(height: 15),
224
225     // BUTTON
226     ElevatedButton(
227       onPressed: fetchAndPredictTweets,
228       style: ElevatedButton.styleFrom(
229         backgroundColor: Colors.deepPurple,
230         foregroundColor: Colors.white,
231         shape: RoundedRectangleBorder(
232           borderRadius: BorderRadius.circular(12),
233         ), // RoundedRectangleBorder
234         padding: EdgeInsets.symmetric(
235           vertical: 15,
236           horizontal: 80,
237         ), // EdgeInsets.symmetric
238         elevation: 5,
239       ),
240       child: isLoading
241         ? CircularProgressIndicator(color: Colors.white)
242         : Text(
243             "Get Predictions",
244             style: TextStyle(fontSize: 18),
245           ), // Text
246     ), // ElevatedButton
247   ],
248 ), // Column
249 ), // Container
250
251   SizedBox(height: 20),
252

```

```

253 // STATUS MESSAGE
254 if (statusMessage.isNotEmpty)
255   Text(
256     statusMessage,
257     textAlign: TextAlign.center,
258     style: TextStyle(
259       fontSize: 16,
260       fontWeight: FontWeight.bold,
261       color: statusMessage.contains("Error")
262         ? Colors.red
263         : Colors.green,
264     ), // TextStyle
265   ), // Text
266
267   SizedBox(height: 20),
268
269 // PREDICTION CARDS
270 if (predictions.isNotEmpty)
271   Column(
272     children: predictions.map((p) {
273       return Container(
274         margin: EdgeInsets.only(bottom: 12),
275         padding: EdgeInsets.all(15),
276         decoration: BoxDecoration(
277           color: Colors.white,
278           borderRadius: BorderRadius.circular(18),
279           boxShadow: [
280             BoxShadow(blurRadius: 8, color: Colors.black12),
281           ],
282         ), // BoxDecoration
283         child: Column(
284           crossAxisAlignment: CrossAxisAlignment.start,
285           children: [
286             Text(
287               p['post'],
288               style: TextStyle(
289                 fontSize: 16,
290                 fontWeight: FontWeight.w600,
291               ), // TextStyle
292             ), // Text
293             Divider(),
294             Text(

```

```

295         "Prediction: ${p['prediction']}",
296         style: TextStyle(
297           color: Colors.deepPurple,
298           fontWeight: FontWeight.bold,
299         ), // TextStyle
300       ), // Text
301       Text(
302         "Probability: ${p['probability'].toStringAsFixed(2)}",
303         style: TextStyle(color: Colors.black54),
304       ), // Text
305     ],
306   ), // Column
307 ); // Container
308   }).toList(),
309 ), // Column
310 ],
311 ), // Column
312 ), // SingleChildScrollView
313 ), // SafeArea
314 ); // Scaffold
315 }
316 }
317

```

11.2. .env

```

1  TWITTER_BEARER_TOKEN=AAAAAAAAAAAAAAAAAGBm5gEAAAAA4dFVmtMusKIXH2D7UoNr8pjY3qo%3DdpI16rtXVhXSNhUED5jddVfm4fg9I13awWwP5sKTPhcso5DUwi
2  API_URL=http://10.252.49.129:5000/predict
3
4
5

```

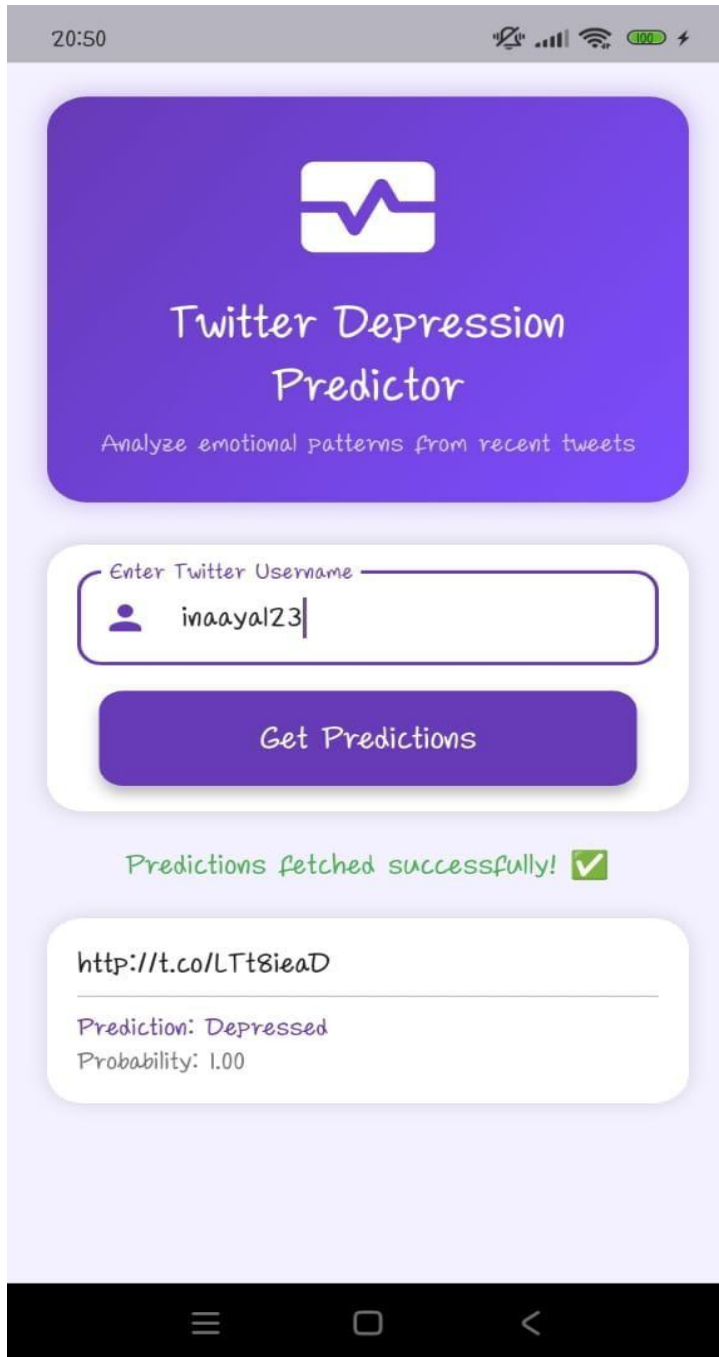
11.3. pubspec.yaml

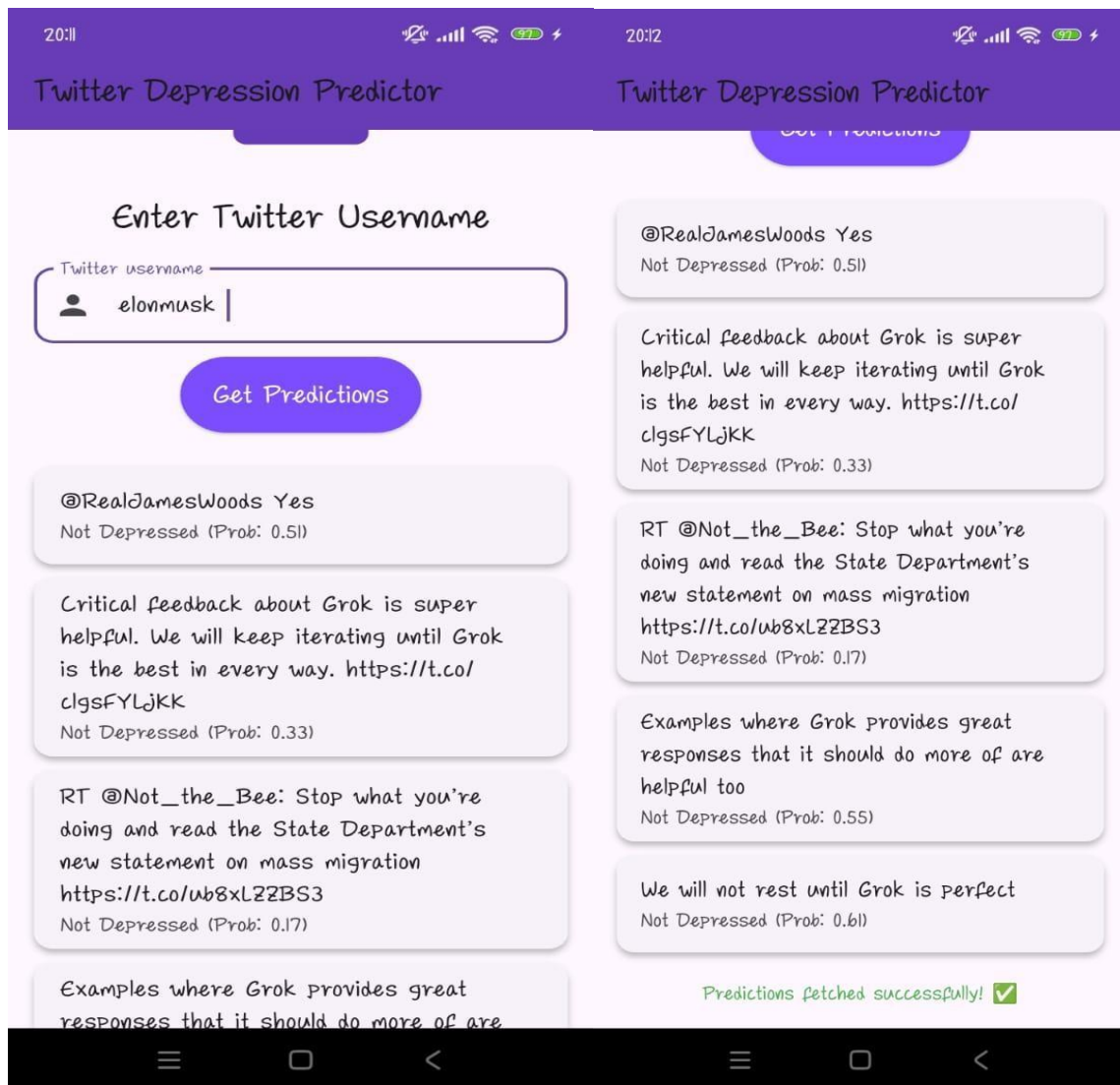
```

1  name: flutter_application_1
2  description: "A new Flutter project."
3  publish_to: 'none'
4  version: 0.1.0
5
6  environment:
7    sdk: ">=3.9.0 <4.0.0"
8
9  dependencies:
10   flutter:
11     sdk: flutter
12   http: ^1.0.0
13   flutter_dotenv: ^5.0.2 # For storing API keys safely
14
15  dev_dependencies:
16   flutter_test:
17     sdk: flutter
18   flutter_lints: ^5.0.0
19
20  flutter:
21    uses-material-design: true
22
23    assets:
24      - assets/.env
25

```

12.Output





External Links and Repositories:

- **Google Drive:** https://drive.google.com/file/d/1g5O_zvpo-ffKJS3akVXOdI8iyMh_0S5p/view?usp=sharing
- **Github:** <https://github.com/aqsariazshaikh/depression-detection-ml>
- **LinkedIn:** https://www.linkedin.com/posts/activity-7404552004645056513-8j1V?utm_source=share&utm_medium=member_desktop&rcm=ACoAAFFds5UBHxeQYYRffgb2_6FDIG0vpNdH0gg

THE END